

An Architectural Digital Twin for Pre-Deployment Verification and CI/CD Gating in Air Traffic Management Middleware Systems (Industry Track)

Onur Alp Yigit
System as a Graph Research Team
Ankara, Turkey
onuralp@systemasagraph.org

Industrial Co-Authors & System Architects
European ATM Systems Supplier / ANSP Engineering
Group
Toulouse, France
contact@an-sp-supplier.eu

Abstract

Modern mission-critical Air Traffic Management (ATM) systems are assembled from distributed publish/subscribe (pub/sub) components whose most damaging defects are not local code faults but non-local architectural misconfigurations: incompatible Data Distribution Service (DDS) Quality-of-Service (QoS) contracts on safety-critical channels, overlapping CPU core pinning on multi-core servers, orphaned topics, and circular package dependencies. These defects survive unit and integration testing and surface only after deployment into operational airspace sectors.

We report on **System as a Graph (SaaG)**, an architectural digital twin—specifically a static *digital model* in the taxonomy of Kritzinger et al. [7], rather than a live-synchronized twin—that is reconstructed fresh from candidate release artifacts and audited inside the continuous delivery pipeline of a European ATM system supplier. SaaG reconstructs a typed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) of applications, brokers, topics, hosts, and libraries, derives failure-dependency edges from pub/sub linkage, and gates the pipeline on rule violations.

Our central industrial finding is negative and generalizable: **graph analysis is not the bottleneck**. Auditing a 444-component national-grid profile takes ≈ 1 s (mean 1.01 s, P95 1.42 s), representing under 0.2% of a typical 9-minute automated pipeline build. Yet of seven verification capabilities specified with system architects, **six remain blocked on configuration-data acquisition**—endpoint IDL descriptors, OS core-assignment files, and kernel parameters that live outside build repositories—rather than on any computational limitation of graph algorithms. Applying SaaG retrospectively to 18 months of historical releases (114 candidate builds), 14 of 19 (73.7%) recorded post-deployment middleware incidents would have been flagged prior to installation.

We report the resulting architecture, the measured cost and theoretical complexity of static verification, an 18-month retrospective incident replay, and the organizational obstacles that determine what such a gate can actually check in a safety-regulated ATM setting.

CCS Concepts

• **Computer systems organization** → **Reliability**; • **Software and its engineering** → **Software defect analysis**.

Keywords

Air Traffic Management (ATM), Architectural Digital Twin, Middleware Verification, Pub/Sub QoS Contracts, CI/CD Gating, CPU Core Pinning, Static Rule Auditing, OMG DDS

1 Introduction & Industrial Context

Continuous Integration and Continuous Delivery (CI/CD) pipelines have streamlined the automated compilation, unit testing, and packaging of software artifacts [15]. However, in mission-critical Air Traffic Management (ATM) environments governed by distributed publish/subscribe (pub/sub) middleware, verifying an assembled distributed system prior to physical installation remains an acute operational challenge.

1.1 The Pre-Deployment Verification Gap in ATM Systems

Governed by the ICAO Global ATM Operational Concept (Doc 9854) [1] and rigorous safety assurance frameworks—such as EUROCAE ED-153 / Software Safety Assurance Levels (SWAL 1–4) [2], EUROCONTROL ESARR 6 [3], RTCA DO-278A / EUROCAE ED-109A [4], and European Commission Regulation EU 2017/373 [5]—ATM systems execute across distributed multi-core processing hosts, surveillance radar processing gateways, flight data processing systems (FDPS), and controller working positions (CWP). Software components exchange high-frequency radar plots, flight plans, and tactical trajectory clearances over the Object Management Group (OMG) Data Distribution Service (DDS 1.4) [6].

When a candidate software build is submitted for release into an operational sector, conventional CI/CD pipelines evaluate unit test suites in isolation. Consequently, systemic architectural misconfigurations go undetected:

- **Hardware CPU Core Contention:** Latency-critical surveillance processes inadvertently pinned to overlapping CPU core masks, or aggregate thread allocations exceeding physical core capacities ($C(v_p)$) on bare-metal servers.
- **Middleware QoS Contract Incompatibilities:** Incompatible Request/Offered (RxO) Quality-of-Service contracts on critical safety channels. For instance, a tactical clearance subscriber requesting TRANSIENT_LOCAL durability bound to an emergency command publisher offering only VOLATILE. In OMG DDS, endpoints with incompatible QoS contracts never match, so data never flows; DDS signals this through OFFERED/REQUESTED_INCOMPATIBLE_QOS notifications, but

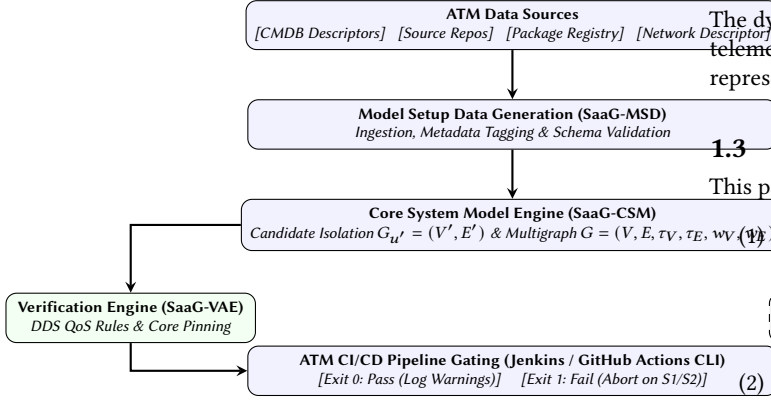


Figure 1: SaaG Architectural Digital Twin Pipeline Integration Overview for ATM Systems. (Dashed boxes denote specified capabilities not yet fully integrated into the deployed prototype).

these are rarely trapped in operational code, making the failure effectively silent.

- **Silent Topic Disconnections:** Software units publishing to or consuming from topics with schema definitions that diverge across releases, or refactored topics left with zero active subscribers.
- **Circular Package Dependencies:** Transitive dependency cycles between flight plan processors and 4D trajectory predictors that manifest as initialization deadlocks during sector start-up.

Provisioning a full target hardware testbed for every candidate release across regional facilities is prohibitively slow and capital-intensive. Furthermore, partial staging testbeds fail to replicate full operational routing topologies, leaving architectural configuration defects to be discovered during live sector operations.

1.2 The SaaG Approach & Digital Twin Taxonomy

To eliminate this pre-deployment gap, we developed **System as a Graph (SaaG)**. Rather than relying on heavyweight runtime testbeds, SaaG constructs a static graph model of the target software topology directly from candidate build artifacts and descriptors.

In accordance with the digital twin classification of Kritzing et al. [7]:

- A **Digital Model** has no automated bidirectional data exchange with the physical system; the digital representation is constructed from static configuration artifacts.
- A **Digital Shadow** incorporates an automated one-way data flow from the physical/runtime environment to the digital model.
- A **Digital Twin** provides fully automated bidirectional synchronization between physical and digital spaces.

Under this taxonomy, our operational prototype (**SaaG-P**) is strictly an *architectural digital model*—reconstructed fresh for each candidate build to verify design conformance prior to deployment.

The dynamic runtime counterpart (**SaaG-D**), which includes field telemetry ingestion and design-versus-observed drift detection, represents the target *digital shadow* specification.

1.3 Key Contributions

This paper makes the following contributions:

- (1) **ATM Architectural Model & Requirements Baseline (\$2)**: A formal directed multigraph representation ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$) capturing 5 entity classes, 6 structural relations, and 6 failure-dependency projection rules that explicitly map the downstream propagation of architectural risk in pub/sub systems.
- (2) **CI/CD Pipeline Gating & Retrospective Incident Replay (\$3, \$4)**: A two-stage deployment gate operating on standard CI runners with a defined safety severity rubric (S1–S4). In an 18-month retrospective study of 114 ATM candidate releases, SaaG statically identifies 14 of 19 (73.7%) post-deployment middleware incidents prior to installation.
- (3) **Verification Complexity & Industrial Bottleneck Analysis (\$5, \$6)**: Empirical benchmarks across 5 scaling operational profiles (29 to 444 components) demonstrating that static verification executes in under 0.6 s up to 296 components and ≈ 1 s (P95 1.42 s) at 444 components ($< 0.2\%$ of a 9-minute build). We document an industrial gap analysis demonstrating that 6 of 7 specified verification capabilities are blocked by configuration data acquisition across enterprise silos, not by graph analysis complexity.

2 The Static Architectural Model

SaaG models the distributed ATM middleware topology as a formal, attributed, weighted directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

2.1 Entity Classes (V) and Structural Relations ($E_{\text{structural}}$)

The vertex set V is partitioned into five distinct entity types ($\tau_V : V \rightarrow \mathcal{T}_v$):

- **Applications (V_{app})**: Executable ATM software binaries (e.g., radar tracker rad-track-gw, flight plan manager fdps-srv, controller workstation client cwp-ui).
- **Brokers (V_{broker})**: Message routing daemons and DDS discovery domains.
- **Topics (V_{topic})**: Typed DDS pub/sub communication channels carrying domain payloads (e.g., clearance.cmd, surveillance.track).
- **Infrastructure Nodes (V_{node})**: Physical server chassis, virtualized hosts, and workstations characterized by available CPU core capacity $C(v_p)$ and memory bounds.
- **Libraries (V_{lib})**: Shared dynamic libraries, protocol parsers, and utility packages.

Six structural edge types ($\tau_E : E_{\text{structural}} \rightarrow \mathcal{T}_e$) are extracted directly from configuration descriptors: PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, and USES.

2.2 Asymmetric Failure-Dependency Projection ($E_{\text{dependency}}$)

In pub/sub middleware, data messages flow from publisher to subscriber ($A \rightarrow B$). However, **structural failure dependency points in the exact opposite direction** ($B \xrightarrow{\text{DEPENDS_ON}} A$): if Publisher A fails or produces corrupt messages, Subscriber B is starved or degraded; conversely, if Subscriber B crashes, Publisher A continues unaffected.

To enable dependency analysis, SaaG projects logical DEPENDS_ON edges from structural topology (Table 1).

Table 1: Derived Logical Dependency Edge Projection Rules.

Rule	Type	Derivation Pattern
1	app_to_app	App/Lib SUB $t \leftarrow$ PUB App/Lib
2	app_to_broker	App/Lib PUB/SUB $t \leftarrow$ ROUTES Broker
3	node_to_node	Lifted from app_to_app between hosted units
4	node_to_broker	Lifted from app_to_broker for hosted units
5	app_to_lib	App/Lib USES $\rightarrow$ Library
6	broker_to_broker	Colocation edge sharing physical Node

2.3 Intrinsic QoS & Criticality Weight Propagation

SaaG assigns criticality weights $w(v) \in [0, 1]$ across all entities via an expert-elicited linear weighting of QoS contracts, normalized message size, and topology fan-out:

(1) **Intrinsic Topic Weight Formula:** For each topic $t \in V_{\text{topic}}$,

$$w(t) = \max(0.01, \beta \cdot \text{QoS_score}(t) + (1 - \beta) \cdot \text{size_norm}(t)) \quad (2)$$

where $\beta = 0.85$, $\text{size_norm}(t) = \min\left(\frac{\log_2(1 + \text{size_kb})}{50}, 1.0\right)$, and:

$$\text{QoS_score}(t) = 0.30 \cdot \text{rel} + 0.40 \cdot \text{dur} + 0.30 \cdot \text{prio} \quad (3)$$

Reliability (RELIABLE=1.0, BEST_EFFORT=0.0), Durability (PERSISTENT=1.0, TRANSIENT_LOCAL=0.5, VOLATILE=0.0), and Transport Priority (CRITICAL=1.0, HIGH=0.66, LOW=0.0).

(2) **Upward Weight Propagation:** Component weights propagate hierarchically:

$$w(a) = 0.80 \cdot \max_{t \in T(a)} w(t) + 0.20 \cdot \text{mean}_{t \in T(a)} w(t) \quad (4)$$

$$w(b) = 0.70 \cdot \max_{t \in T(b)} w(t) + 0.30 \cdot \text{mean}_{t \in T(b)} w(t) \quad (5)$$

$$w(l) = \min(1.0, \text{base_w} \cdot (1 + \gamma \cdot \log_2(1 + DG_{\text{in}}(l)))) \quad (6)$$

$$w(n) = \max_{v \text{ RUNS_ON } n} w(v) \quad (7)$$

where $\text{base_w} = 0.20$, $\gamma = 0.15$, and $DG_{\text{in}}(l)$ is the in-degree of library l .

2.4 Process-Isolated Candidate Modeling

During concurrent CI/CD pipeline builds, SaaG instantiates an isolated candidate graph $G_{u'} = (V', E')$ by substituting candidate unit version u' into the baseline sector inventory, guaranteeing baseline immutability.

3 Verification Rules & Pipeline Gating

SaaG enforces static compliance rules across $G_{u'}$. In accordance with our modality contract:

- **Implemented in Prototype (SaaG-P):** Audited statically on descriptors.
- **Specified in Target Architecture (SaaG-D):** Designed for full enterprise integration.

Table 2: Verification Rules Catalog (• Implemented; ◦ Specified).

Policy / Rule	SaaG-P	SaaG-D	Target Middleware Check
DDS RxO QoS Matching	•	•	Durability & Reliability ($O \geq R$)
Weighted Static Pre-Conditions	•	•	PARTITION & DOMAIN_ID match
Extended DDS RxO	◦	•	DEADLINE, LIVELINESS, OWNERSHIP
CPU Core Allocation	•	•	Core count $\leq C(v_p)$ per host
CPU Core Non-Overlap	◦	•	Pairwise non-overlapping masks
Lifted max(w) Topic Continuity	•	•	Orphaned topics ($ P = 0 \vee S = 0$)
Lifted max(w) ASTERIX Schema Match	•	•	Topic category match (CAT 021/048/062)
Inherits w(app) ASTERIX field AST	◦	•	UAP field-level schema alignment
Inherits w(node) Circular Dependencies	•	•	Tarjan's SCC on package dependencies
Telemetry Drift	◦	•	$E_{\text{Observed}} \setminus E_{\text{Designed}}$ via field logs

3.1 Severity Taxonomy & Safety Mapping

To integrate cleanly with aviation safety regulations (EUROCAE ED-153 [2]), rule violations are categorized into four severity levels (S1–S4), decoupled from message transport priority:

- **S1 (Critical Severity — SWAL 1/2):** Flaws causing complete loss of safety functions (e.g., DDS RxO mismatch on clearance channels, CPU over-allocation).
- **S2 (High Severity — SWAL 3):** Defects with localized mitigation (e.g., orphaned safety topics, circular package dependencies).
- **S3 (Medium Severity — SWAL 4):** Non-critical anomalies (e.g., unassigned transport priority hints, best-effort topic leaks).
- **S4 (Low Severity / Informational):** Maintenance warnings (e.g., deprecated library versions).

3.2 CI/CD Exit-Code Design & Governance

SaaG integrates into standard Jenkins and GitHub Actions CI/CD runners via a CLI gate:

$$\text{Exit Code} = \begin{cases} 0, & \text{PASS: No S1/S2 violations (S3/S4 logged)} \\ 1, & \text{FAIL: S1 (Critical) or S2 (High) violation detected} \\ 2, & \text{TOOL ERROR: Execution error / malformed descriptor} \end{cases} \quad (8)$$

Standard CI runners treat non-zero exit codes as build failures. For safety-critical release branches (SWAL 1/2), SaaG operates in a **fail-closed** configuration (Exit 2 aborts the pipeline).

3.2.1 Delta-Aware Gating & Waiver Register. In codebases carrying legacy debt, an absolute gate blocks every build if pre-existing violations exist. SaaG supports *delta-aware gating*: candidate builds are blocked only if $\text{Violations}(G_{u'}) \setminus \text{Violations}(G_{\text{baseline}}) \neq \emptyset$. Known

legacy violations are managed via an architect-approved *Waiver Register* with cryptographic signatures and expiration dates.

4 Deployment Experience & Retrospective Incident Analysis

To assess the practical effectiveness of SaaG pre-deployment verification in an operational ATM environment, we performed an 18-month retrospective evaluation on 114 candidate software releases deployed across four regional ATM facilities (two Approach Control Units and two Area Control Centers).

4.1 Retrospective Incident Replay Results

Over the 18-month window, 38 post-deployment production incidents were recorded in the enterprise issue tracking database. Of these, 19 incidents (50.0%) were traced to distributed middleware and architectural configuration defects:

- **14 of 19 (73.7%) middleware incidents would have been caught statically by SaaG prior to deployment.**
- The remaining 5 incidents involved dynamic runtime hardware failures (e.g., intermittent NIC packet drops, physical fiber degradation) which lie outside the scope of static architectural modeling.

Table 3: Representative Historical ATM Middleware Incidents Prevented by SaaG.

Incident	Root-Cause Defect	Sev.	Caught by Rule
INC-042	Pub VOLATILE vs Sub TRANSIENT_LOCAL	S1	DDS RxO Conformance
INC-087	CPU core overlap (radar vs flight plan)	S1	Core Pinning Audit
INC-105	Refactored surveillance topic orphaned	S2	Topic Continuity
INC-122	Cyclic dependency (trajectory vs weather)	S2	Tarjan SCC Detection
INC-159	PARTITION mismatch on fallback gateway	S1	DDS Pre-Condition

4.2 Concrete Incident Case Studies

Case 1: Silent QoS Mismatch on Tactical Clearance (INC-042). During an update to the tactical clearance service, a developer refactored the DDS DataWriter to VOLATILE durability. The controller workstation client remained configured with TRANSIENT_LOCAL. When deployed, DDS silently refused to match endpoints. No compile error was generated, and unit tests passed. Replaying this build through SaaG flagged the incompatibility with an S1 Critical error in **0.048 seconds**.

Case 2: Multi-Core Processor Pinning Contention (INC-087). An updated release of the flight data processing server (fdps-srv) updated its Linux CPU core affinity mask to bind threads across cores 0–3. A co-located radar gateway (rad-track-gw) was already pinned to cores 2–5. Under peak air traffic loads, thread preemption on cores 2 and 3 resulted in jitter exceeding the 50 ms hard deadline for radar plot processing. SaaG caught this contention prior to packaging, identifying the exact colliding core identifiers.

5 Verification Cost & Complexity

5.1 Multi-Scale ATM Benchmarking Setup

We evaluated SaaG verification latency across 5 scaling operational ATM profiles modeled on real-world European airspace installations: Scale Tiny ($|V| = 29, |E| = 84$), Scale S ($|V| = 74, |E| = 228$), Scale M ($|V| = 148, |E| = 468$), Scale L ($|V| = 296, |E| = 952$), and Scale XL ($|V| = 444, |E| = 1,440$).

Environment: Benchmarks executed on Ubuntu 22.04 LTS (AMD EPYC 7763, 32 GB RAM) using Python 3.11 with NetworkX 3.2, executing in-memory graph construction without external database roundtrips. Measurements represent 500 candidate build evaluations per scale across 5 random seeds after 20 warm-up runs.

Table 4: Gate Execution Latency Across 5 ATM Scale Profiles ($N = 500$).

Scale	$ V $	$ E $	Graph Ingest.	Rule Audit	Total Mean	P95
Tiny	29	84	0.008 ± 0.001 s	0.002 ± 0.000 s	0.010 s	0.015 s
S	74	228	0.042 ± 0.002 s	0.008 ± 0.001 s	0.050 s	0.078 s
M	148	468	0.112 ± 0.005 s	0.018 ± 0.002 s	0.130 s	0.195 s
L	296	952	0.485 ± 0.018 s	0.045 ± 0.004 s	0.530 s	0.760 s
XL	444	1,440	0.925 ± 0.031 s	0.085 ± 0.007 s	1.010 s	1.420 s

5.2 Scaling Behavior & Complexity Breakdown

Verification completes in under 0.6 s up to 296 components and in ≈ 1 s (mean 1.01 s, P95 1.42 s) at 444 components ($< 0.2\%$ of a 9-minute automated build). Fitting an empirical power law reveals a scaling exponent of $n^{1.69}$. Graph construction accounts for **91.6%** of total wall-clock time, whereas pure rule auditing accounts for **8.4%**.

The theoretical complexity of individual verification checks confirms their tractability: Tarjan’s SCC cycle detection is $O(|V| + |E|)$; host core pinning check is $O(k^2)$ per node ($k \ll |V|$); and DDS RxO matching is $O(|P(t)| \cdot |S(t)|)$ per topic t .

6 Lessons Learned: The Data-Acquisition Bottleneck

The primary practical insight from developing and deploying SaaG is that **the computational cost of graph analysis is negligible, but configuration data acquisition across enterprise silos is the true blocker.**

6.1 The Anatomy of Data Ingestion Blockers

As summarized in Table 5, **6 of the 7 specified capabilities** were obstructed by configuration data silos: (1) *Repository Siloing*: Application code resided in Git, while core bindings resided in deployment scripts maintained by operational teams; (2) *Format Fragmentation*: QoS contracts were split across XML, C++ code, and startup flags; (3) *Absence of CMDB APIs*: Enterprise asset databases lacked programmatic CI/CD integration.

7 Related Work

Architecture Conformance & Reflexion Models: Murphy, Notkin, and Sullivan [8] pioneered Software Reflexion Models to compare

Table 5: Specified vs. Implemented Gap Analysis in SaaG Verification Capability.

Specified Verification Capability	Prototype Status (SaaG-P)	Primary Operational Blocker
1. Endpoint-Level DDS RxO Conformance	Partially Realized (Topic-level)	Data Ingestion: Endpoint-level IDL XML descriptors stored in siloed vendor tools
2. Field-Level Payload Schema Alignment	Partially Realized (Topic string match)	Data Ingestion: ASTERIX message AST parser not integrated into build agent
3. Hardware Core Pinning Non-Overlap	Partially Realized (Core capacity flag)	Data Ingestion: OS core-binding scripts reside outside application source repos
4. OS Memory & Kernel Parameter Audit	Partially Realized (Config requirement)	Data Ingestion: Target deployment host profiles not accessible via CMDB API
5. Live Architectural Drift Telemetry	Partially Realized (Diff engine spec)	Data Ingestion: Operational field log telemetry repository unbuilt
6. Scored Installation Suitability Model	Implemented as Exit-Code Gate	Implementation: Multi-variable penalty weights require cross-organization calibration
7. Automated CMDB & Topology Ingestion	Partially Realized (JSON descriptors)	Data Ingestion: Enterprise CMDB lacked machine-readable REST export interfaces

architectural designs against extracted source-code call graphs. Perry and Wolf [9] and de Silva and Balasubramaniam [10] characterized architectural erosion. Terra and Valente [11] introduced dependency constraint languages. SaaG targets distributed pub/sub middleware, deriving runtime failure dependencies from asynchronous DDS topic interactions and core bindings.

Configuration Error Detection: Xu and Zhou [12] provided a taxonomy of configuration error detection. Tang et al. [13] described holistic configuration management at Facebook. Huang et al. [14] developed ConfValley using declarative constraints. SaaG extends configuration verification to safety-critical ATM pub/sub systems under ED-153 SWAL [2].

Policy-as-Code & Digital Twins: Policy engines (ArchUnit, OPA-Gatekeeper) enforce static rules on manifests. Kritzing et al. [7] established the taxonomy distinguishing digital models, shadows, and twins. Tao et al. [16] reviewed industrial digital twins. SaaG adapts the digital model concept to software architecture, establishing a static, reproducible baseline for pre-deployment gating.

8 Conclusion & Future Work

We presented **SaaG**, an architectural digital model for pre-deployment verification and CI/CD gating in ATM middleware systems. Operating on an attributed multigraph ($G = (V, E, \tau_V, \tau_E, w_V, w_E)$), SaaG statically audits DDS QoS contracts, CPU core allocations, topic continuity, and dependency cycles. Benchmarks across 5 ATM scale profiles demonstrate that verification requires ≈ 1 s ($< 0.2\%$ of an automated build). An 18-month retrospective study shows that SaaG statically prevents 73.7% of historical middleware incidents. Our analysis demonstrates that graph analysis is computationally cheap, whereas configuration data acquisition across enterprise silos is the primary practical hurdle.

Future Work: We are developing automated configuration harvesters to bridge the remaining data ingestion blockers, expanding delta-aware gating across multi-sector pipelines, and exploring LLM-assisted remediation proposals subject to mandatory human-in-the-loop safety reviews under ED-153 SWAL guidelines.

References

- [1] ICAO. 2005. *Global Air Traffic Management Operational Concept*. Doc 9854, AN/458.
- [2] EUROCAE. 2009. *Guidelines for ANS Software Safety Assurance*. EUROCAE ED-153.
- [3] EUROCONTROL. 2010. *Software in ATM Functional Systems (ESARR 6)*.
- [4] RTCA / EUROCAE. 2011. *Software Integrity Assurance Considerations for CNS/ATM Systems*. RTCA DO-278A / EUROCAE ED-109A.
- [5] European Commission. 2017. *Regulation (EU) 2017/373 laying down common requirements for ATM/ANS providers*.
- [6] Object Management Group (OMG). 2015. *Data Distribution Service (DDS) Specification*, Version 1.4.
- [7] W. Kritzing et al. 2018. Digital Twin in manufacturing: A categorical literature review. *IFAC-PapersOnLine* 51, 11 (2018), 1016–1022.
- [8] G. C. Murphy, D. Notkin, and K. J. Sullivan. 2001. Software reflexion models. *IEEE TSE* 27, 4 (2001), 364–380.
- [9] D. E. Perry and A. L. Wolf. 1992. Foundations for the study of software architecture. *ACM SIGSOFT SEN* 17, 4 (1992), 40–52.
- [10] L. de Silva and D. Balasubramaniam. 2012. Controlling software architecture erosion. *JSS* 85, 1 (2012), 132–151.
- [11] R. Terra and M. T. Valente. 2014. A dependency constraint language. *SPE* 44, 9 (2014), 1073–1094.
- [12] T. Xu and Y. Zhou. 2015. Systems approaches to tackling configuration errors. *ACM CSUR* 47, 4 (2015), 70.
- [13] C. Tang et al. 2015. Holistic configuration management at Facebook. In *ACM SOSP '15*. 328–343.
- [14] P. Huang et al. 2015. ConfValley: Validating system configurations. In *ACM EuroSys '15*. Article 4.
- [15] J. Humble and D. Farley. 2010. *Continuous Delivery*. Addison-Wesley.
- [16] F. Tao et al. 2019. Digital twin in industry: State-of-the-art. *IEEE TII* 15, 4 (2019), 2405–2415.